

# 30-Minute Student Task

## Build Your Own Rule-Based Python Chatbot - Solved

**Name:** Darsi Gopi

**Reg No:** 99230041179

**Department:** CSE (AI & ML)

**College:** Kalasalingam University of Research and Education

**Date:** 11-09-26

Based on the requirements and target flow in the provided task sheet.

### Task Summary

The task is to build a simple rule-based conversational chatbot that continuously interacts with the user, recognizes different messages, gives appropriate responses, and exits when the user types "bye" or "exit." The required flow is:

**INPUT → PROCESS → RESPOND → REPEAT → EXIT**

### Requirement Checklist

| Requirement                                        | How it is satisfied                                     |
|----------------------------------------------------|---------------------------------------------------------|
| Ask the user's name before the main conversation.  | Implemented with input() before the loop.               |
| Use while True.                                    | Used to keep the conversation running.                  |
| Use input() to receive messages.                   | Used for the user's name and every message.             |
| Clean input using .lower().strip().                | Applied to every conversation message.                  |
| Recognize at least two greetings.                  | Recognizes hello and hi, including variations using in. |
| Recognize at least three questions/topics.         | Covers Python, weather, help, and time-related topics.  |
| Use if/elif/else.                                  | The chatbot uses an if/elif/else decision structure.    |
| Use break for bye or exit.                         | The loop breaks when the cleaned input is bye or exit.  |
| Use the user's name in at least one response.      | The greeting and goodbye use the user's name.           |
| Useful response for unknown messages using else    | Unknown input receives a helpful fallback response.     |
| Challenge: use in operator for flexible greetings. | Recognizes phrases such as 'hello bot' and 'hey hello'. |

### Complete Python Solution

```
print("Welcome to the Rule-Based Python Chatbot!")
name = input("What is your name? ").strip()
print(f"Nice to meet you, {name}!")
print("You can say hello, ask about Python, ask about the weather, ")
print("ask for help, or type 'bye' or 'exit' to leave.")

while True:
    message = input("You: ").lower().strip()
    if message == "bye" or message == "exit":
        print(f"Goodbye, {name}! Have a great day.")
        break
    elif "hello" in message or "hi" in message:
        print(f"Hello, {name}! How can I help you?")
    elif "python" in message:
        print("Python is a beginner-friendly programming language used for web development, automation, data analysis, and more.")
    elif "weather" in message:
        print("I cannot check live weather, but you can use a weather app or website for the current forecast.")
    elif "help" in message:
        print("You can greet me, ask about Python, ask about the weather, or type 'bye' or 'exit' to end the chat.")
    elif "time" in message:
        print("I cannot access your local clock in this simple rule-based chatbot.")
    else:
        print("Sorry, I do not understand that yet. Try saying hello, asking about Python or weather, asking for help, or saying bye or exit.")
```

## How the Chatbot Works

**INPUT:** The program first asks the user for a name, then repeatedly uses `input()` to receive messages.

**PROCESS:** Each message is cleaned with `.lower().strip()`, so inputs such as HELLO and hello are treated consistently.

**RESPOND:** The `if/elif/else` statements compare the cleaned message with known rules and print the appropriate response.

**REPEAT:** `while True` keeps the conversation running after each response.

**EXIT:** If the user enters `bye` or `exit`, the chatbot prints a goodbye message and uses `break` to leave the loop.

## Test Cases and Expected Results

| Test               | Input               | Expected Result                                         |
|--------------------|---------------------|---------------------------------------------------------|
| Name               | Darsi Gopi          | Nice to meet you, Darsi Gopi!                           |
| Greeting           | hello               | Hello, Darsi Gopi! How can I help you?                  |
| Uppercase greeting | HELLO               | Hello, Darsi Gopi! How can I help you?                  |
| Second greeting    | hi                  | Hello, Darsi Gopi! How can I help you?                  |
| Flexible greeting  | hello bot           | Hello, Darsi Gopi! How can I help you?                  |
| Flexible greeting  | hey hello           | Hello, Darsi Gopi! How can I help you?                  |
| Question/topic     | What is Python?     | A short explanation of Python is displayed.             |
| Question/topic     | Tell me the weather | The chatbot explains that it cannot check live weather. |
| Help               | help                | The chatbot lists the supported commands/topics.        |
| Unknown message    | I like pizza        | A useful fallback message is displayed.                 |
| Exit               | bye                 | A goodbye message is displayed and the loop ends.       |
| Exit               | exit                | A goodbye message is displayed and the loop ends.       |

## Sample Demonstration

```
Welcome to the Rule-Based Python Chatbot!
What is your name? Darsi Gopi
Nice to meet you, Darsi Gopi!
You: HELLO
Hello, Darsi Gopi! How can I help you?
You: What is Python?
Python is a beginner-friendly programming language used for web development, automation, data analysis, and AI.
You: hello bot
Hello, Darsi Gopi! How can I help you?
You: I like pizza
Sorry, I do not understand that yet. Try saying hello, asking about Python or weather, asking for help, or typing 'bye'
You: bye
Goodbye, Darsi Gopi! Have a great day.
```

## Why This Solution Meets the Task

This solution follows the task sheet rather than relying on a copied chatbot example. It uses the requested conversational loop, input cleaning, multiple greetings, multiple topics, `if/elif/else` decision making, an exit condition with `break`, the user's name, and a useful unknown-message response. It also implements the challenge by using the `in` operator so greeting variations such as "hello bot" and "hey hello" are recognized.

## Final Target Flow

**INPUT → PROCESS → RESPOND → REPEAT → EXIT**

The task sheet specifically identifies this as the target flow and asks the student to test greetings, uppercase input, questions, an unknown message, and `bye/exit` before preparing the demonstration.